

01-实验1：RISC-V引导与裸机启动

实验1：RISC-V 引导与裸机启动 (Bare-metal Startup)

实验目标

本实验旨在打破“C 语言程序总是从 `main` 函数开始”的思维定势。在没有操作系统的“裸机 (Bare-metal)”环境下，你将抛弃所有标准库，亲手编写底层引导代码，为 C 语言的运行创造基础条件，并最终利用物理内存地址直接操控硬件，在屏幕上输出 “Hello OS”。

理论回顾：真机启动 vs QEMU 启动

在理论课中，一台物理计算机的启动 (Boot) 通常包含三个漫长的阶段：

1. **硬件自检 (POST)**：主板通电，检查 CPU、内存、外设状态是否正常。
2. **引导加载 (Bootloader)**：运行 BIOS/UEFI 或 OpenSBI 等固件，它们从硬盘主引导记录中读取 Bootloader (如 GRUB)。
3. **内核加载**：Bootloader 驱动硬盘，将操作系统内核 (Kernel) 从硬盘复制到内存，最后跳转到内核入口。

为什么我们在实验中感觉不到前面的步骤？

因为在这个实验中，QEMU 模拟器充当了“物理魔法”的角色。在启动 QEMU 时的参数包含 `-bios none -kernel kernel.elf`：

- `-bios none`：我们告诉 QEMU 不加载任何底层基础固件（如 OpenSBI），我们要让 CPU 直接处于最原始、特权级最高的机器模式 (Machine Mode)。
- `-kernel kernel.elf`：QEMU 绕过了极其复杂的“驱动硬盘读文件”的过程，直接开启上帝视角，把我们编译好的内核可执行文件“凭空”铺到了虚拟物理内存的 `0x80000000` 地址。

因此，硬件自检和操作系统的早期硬盘引导步骤被 QEMU 省略了。但内核初始化并没有省略。本实验要写的每一行代码（设置栈底 `sp`、配置串口输出），正是内核初始化的“破冰第一步”。

理论与实践的鸿沟：启动的三大难题

在开始写代码前，我们要明白：当计算机刚通电点亮时，面临的是一片混沌的世界。要在这样的环境下跑起 C 语言，你必须亲自越过三道鸿沟：

1. **CPU 的迷茫**：通电后，CPU 怎么知道去内存的哪个绝对位置找第一条指令？
2. **C 语言无法运行**：C 语言的局部变量和函数调用机制必须要有一块内存作为“栈 (Stack)”，而刚开机时，硬件根本没有“栈”的概念。
3. **没有屏幕输出**：不能使用 `#include <stdio.h>` 和 `printf`。我们要怎么让屏幕显示字符？

本实验的任务线，就是为了逐一击破这三个难题。

任务指南与实施步骤

任务1：控制内核在哪里安家（链接脚本）

应对难题 1：CPU 去哪找第一条指令？

在 QEMU 模拟器的“virt”主板规定中，当计算机启动初始化后，它的程序计数器 (PC) 会固定跳到物理内存地址 `0x80000000` 处，开始无脑执行第一条指令。

因此，我们需要写一个链接脚本 (`kernel.ld`)，告诉链接器：“把我的第一行组装好的汇编机器码，严格放在 `0x80000000` 这个地址的开头！”

你的任务：

参考 xv6 的 `kernel/kernel.ld`，在你的项目根目录创建一个 `kernel.ld` 文件。

核心脚手架参考：

```
1 OUTPUT_ARCH( "riscv" )
2 ENTRY( _entry ) /* 告诉链接器：整个程序的真正入口点在 _entry 标签处 */
3
4 SECTIONS
5 {
6     /* 确保内核代码从这个规定的固定物理地址开始 */
7     . = 0x80000000;
8
9     .text : {
10         *(.text .text.*) /* 所有的代码段指令，都紧挨着这里存放 */
11     }
12
13     /* 思考：数据段 (.data) 和 BSS 段 (.bss) 应该放在哪儿？请尝试补充完整。 */
14 }
```

任务2：搭建 C 语言运行的舞台（启动汇编）

应对难题 2：C 语言没有栈怎么跑？

当 CPU 跳入 `0x80000000`，执行到的将是我们用汇编语言写的 `_entry` 标签所在的代码。它的首要任务就是开拓荒芜：随便在内存里划一块区域，给 `sp`（栈指针寄存器）赋个值，将那里作为栈！有了栈，咱们才能大摇大摆地跳入熟悉的 C 语言世界。

你的任务：

在 `kernel/boot/` 目录下创建 `entry.S` 文件。

核心脚手架参考：

```
1     .section .text
2     .global _entry
3     _entry:
4         # 1. 找到事先在内存中划好的连续栈空间的首地址
5         la sp, stack0
6
7         # 2. 我们给栈分配 4KB（即 1页 = 4096 字节）的大小
8         li a0, 4096
9
10        # 3. 计算真正的栈顶。因为栈是由高地址向低地址生长的，所以 sp 必须设在最高处！
11        add sp, sp, a0
12
13        # 4. 栈终于准备好了，现在可以安全跳转到了 C 代码的主函数（不再限定叫 main）
14        call start_main
```

```

15
16     # 5. 万一 C 函数不小心 return 退出了，我们需要一个死循环进行兜底，防止 CPU
    甚至跑到乱码内存里去
17 spin:
18     j spin
19
20     # 下方是静态划分出的一块 4KB 内存数据区，留给栈专门使用
21     .section .bss
22     .align 16
23 stack0:
24     .space 4096

```

任务3：使用内存地址操控硬件（串口驱动）

应对难题 3：没有 `printf` 怎么让屏幕显示内容？

在没有任何库的内核世界里，“屏幕”到底是什么？在 QEMU 中，屏幕就是**串口通信控制器（UART）**。

这里我们要运用一种称为 **MMIO (Memory-Mapped I/O, 内存映射 I/O)** 的魔法：硬件设计时就硬性规定好了，只要 CPU 向特定的神奇内存地址（如 `0x10000000`）写入一个字节，这个数据不会存在内存条里，而是直接发送到了主板的串口电路上，最终在你的终端屏幕上打印出一个字符。

 **你的任务：**

创建 `kernel/driver/uart.c` 文件，实现一个最基础的字符输出函数。

核心脚手架参考：

```

1 // 定义 UART 控制器的基地址（这是 QEMU virt 平台定死的物理地址）
2 #define UART0 0x10000000L
3
4 // 强转为指针。重要陷阱：必须通过 volatile 关键字告诉 C 编译器，
5 // 千万别把这段看似“重复无意义”的赋值代码给优化（删）掉！
6 #define Reg(reg) ((volatile unsigned char *)(UART0 + reg))
7
8 void uart_putc(char c) {
9     // 将字符 c 直接写入硬件寄存器！
10    *Reg(0) = c;
11 }
12
13 // 挑战：实现打印字符串函数
14 void uart_puts(char *s) {
15     // 你需要写一个 while 循环，遍历字符串的每个字符，循环调用 uart_putc 直到遇到 '\0' 结束符
16 }

```

任务4：拼凑主线并通电试运行

现在有了栈，也掌握了控制硬件发消息的方法，是时候让主心骨登场，将所有的零件组装起来供电了。

🔧 你的任务：

1. 创建 C 逻辑所在文件 `kernel/boot/main.c`：

```
1 // 声明在其他文件中实现的驱动函数
2 extern void uart_puts(char *s);
3
4 void start_main() {
5     uart_puts("Hello OS from RISC-V Bare-metal!\n");
6
7     // 操作系统作为一个服务程序，永远不应该自己退出，进入死循环待命
8     while(1);
9 }
```

2. 为了把 `.S` 和 `.c` 用前面写的 `.ld` 脚本缝合在一起，并启动虚拟机给内核上电，在终端执行以下指令（建议你把它写进 `Makefile` 里）：

```
1 # 执行交叉编译与链接，生成纯净的裸内核文件 kernel.elf
2 # 中间的 -nostdlib -fno-builtin 是在告诉编译器：不要链接任何 C 语言自带的标准库！
3 riscv64-unknown-elf-gcc -nostdlib -fno-builtin -mcmodel=medany -T kernel.ld kernel/boot/entry.S kernel/driver/uart.c kernel/boot/main.c -o kernel.elf
4
5 # 放入 QEMU 烧箱启动（没有 BIOS，直接加载我们自己写的 kernel.elf）
6 qemu-system-riscv64 -machine virt -bios none -kernel kernel.elf -nographic
```

避坑与故障排除策略 (Debugging)

如果屏幕一片死寂，不要慌，这是写操作系统最享受的时光。此时没有任何编译器报错，代码逻辑死在了黑盒子里，你需要像侦探一样排查：

分段定位法：

1. 是否卡在了第一行汇编代码 `_entry` 处？

用 GDB 调试看看程序的第一条指令是否真的落地在 `0x80000000` ！

(新终端输入 `qemu-system-riscv64 -machine virt -bios none -kernel kernel.elf -nographic -s -S` 挂起系统，然后再开一个终端输入 `gdb-multiarch kernel.elf` 连接调试)

2. C 语言的栈是否分配成功？

在 GDB 下单步进入汇编 (`si`)，在执行到 `call start_main` 之前，停下来用命令 `info registers sp` 检查一下当前栈指针。看看 `sp` 是不是等于 `stack0` 的地址加上了 `4096`？如果 `sp` 依然是 0，说明前面的汇编加法没写对，肯定进不了 C 语言世界。

3. 是不是打印功能写错了？

如果 GDB 单步跟踪发现成功切入进 `start_main` 里面去了，却没有在终端打印出任何东西。请去检查 `uart_puts` 里向指针写入的最终基地址是不是手误写错了（是否是 `0x10000000`）。

进阶实验验收

实验完成后需能解答以下问题：

1. 请观察 `entry.S` 的代码，如果不给 `sp` 赋值，直接就 `call start_main`，QEMU 终端会发生什么样的运行崩溃或奇怪现象？请从原理解释为何会这样。
2. 我们在 C 语言直接写指针赋值 `*Reg(0) = c;` 就能精准控制主板外部硬件。在宏定义指针时，如果你漏加了 `volatile` 关键字，当你将来用 gcc 的 `-O3` 开启高级优化时，这段串口发送代码可能会被编译器怎么“自作聪明”地修改掉，导致什么严重的后果？